[0] ĐỌC CODE TRƯỚC (bắt buộc)

bash
```
find . -type f \( -name "*.js" -o -name "*.ts" \) | grep -v node_modules | sort
cat package.json
cat src/index.js 2>/dev/null || cat src/app.js 2>/dev/null || cat src/server.js
ls src/routes/ && cat src/routes/*.js 2>/dev/null | head -200
cat src/controllers/*.controller.js 2>/dev/null | head -200
cat src/config/db.js 2>/dev/null || cat src/database/connection.js
cat src/utils/response.js 2>/dev/null || cat src/helpers/response.js
cat src/middlewares/auth.middleware.js 2>/dev/null || cat src/middlewares/auth.js
```

→ Ghi nhớ: framework, ORM/raw-query, response format, auth middleware, file naming convention.

[1] CONVENTIONS (áp dụng toàn bộ)

- **Response**: `{success:bool, data?, message?, pagination?}`
- **Pagination**: params `page`(def:1) + `limit`(def:20) → `{current_page, per_page, total_items, total_pages}`
- **Auth**: Bearer JWT → `req.user = {user_id, roles:string[]}`
- **Seller store**: `SELECT store_id FROM user_stores WHERE user_id=@uid AND store_member_role='OWNER' AND is_active=1`
- **Soft delete**: set `deleted_at=SYSUTCDATETIME()` hoặc `status='INACTIVE'`
- **SQL Server**: dùng `@param`, `OFFSET x ROWS FETCH NEXT y ROWS ONLY`
- **Comment code**: tối thiểu — chỉ comment logic không tự giải thích được

GROUP 13: Messaging (44–48)

Auth: Customer hoặc Seller tùy API

44 · List My Conversations `GET /api/v1/conversations` | Customer, Seller

- Customer: `WHERE c.customer_id=@uid`
- Seller: `WHERE c.store_id=@seller_store_id`

sql
```
SELECT c.*, last_msg.message_content AS last_message_preview, last_msg.created_at
AS last_message_at,
    s.store_name, s.logo_url, u.full_name AS customer_name, u.avatar_url AS
customer_avatar
FROM conversations c
JOIN stores s ON c.store_id=s.store_id
```

```
JOIN users u ON c.customer_id=u.user_id
OUTER APPLY (SELECT TOP 1 message_content, created_at FROM messages WHERE
conversation_id=c.conversation_id ORDER BY created_at DESC) last_msg
WHERE [condition] ORDER BY last_msg.created_at DESC
```

→ **200**: `{conversations:[{conversation_id, store|customer info, last_message_preview, last_message_at, status}]}`

45 · Create Conversation `POST /api/v1/conversations` | Customer

Body: `{store_id, order_id?}`

- Check store tồn tại và ACTIVE
- Nếu đã tồn tại (unique constraint): trả **200** với conversation hiện tại
- Chưa có: INSERT conversations

→ **201** (mới) hoặc **200** (đã có): conversation object

46 · Get Messages `GET /api/v1/conversations/:id/messages` | Customer, Seller

Params: `page`, `limit`

- Verify participant: `c.customer_id=@uid` OR `c.store_id=@seller_store_id`

```
sql
SELECT m.*, u.full_name AS sender_name, u.avatar_url AS sender_avatar
FROM messages m JOIN users u ON m.sender_user_id=u.user_id
WHERE m.conversation_id=@cid ORDER BY m.created_at DESC
```

→ **200**: `{messages:[{message_id, sender{user_id,full_name,avatar_url}, message_content, is_read, created_at}], pagination}`

47 · Send Message `POST /api/v1/conversations/:id/messages` | Customer, Seller

Body: `{message_content}`

- Verify participant
- Validate content không rỗng/whitespace
- INSERT messages
- `UPDATE conversations SET last_message_at=NOW()`
- Emit WebSocket `conversation:{id}:new_message` (nếu WS đã setup)

→ **201**: message object

48 · Mark As Read `PATCH /api/v1/conversations/:id/read` | Customer, Seller

- Verify participant
- `UPDATE messages SET is_read=1, read_at=NOW() WHERE conversation_id=@cid AND sender_user_id!=@uid AND is_read=0`

→ **200**: `{message:"Đã đánh dấu đã đọc"}`

CHECKLIST:

- [ ] Tất cả 5 route (44-48) đăng ký đúng router.
- [ ] Auth middleware linh hoạt: Có thể là Customer HOẶC Seller (kiểm tra Participant).
- [ ] Parameterized query.
- [ ] Validate dữ liệu gửi lên (chống gửi tin nhắn rỗng).
- [ ] Đảm bảo tính nhất quán dữ liệu khi Update `last_message_at` của bảng Conversations sau khi Send Message.
- [ ] Response format nhất quán.